The Curious Coder

Part II - Beyond the Basics



FREE SAMPLE

The Curious Coder

Part II — Beyond the Basics

Part II moves from single statements to complete programs: loops, collections, functions, files, modules, and a finished Task Management Application. This sample gives you the first two chapters in full. Part II assumes you have finished Part I or already know its material.

INCLUDED IN THIS SAMPLE

The cover, copyright page, author page, table of contents, introduction, and Chapters 11–12: Loops and Repetition, and Controlling Loops and Solving Repetitive Problems.

GOOD TO KNOW

- The table of contents lists the full book. Its page numbers refer to the complete edition, and only the links for chapters in this sample are active.
- This sample may be shared in its complete, unmodified form. The full book remains protected by copyright.

© 2026 Md Nasrullah. All rights reserved.

Copyright

The Curious Coder — Part II: Beyond the Basics
First Edition

Copyright © 2026 Md Nasrullah. All rights reserved.

Terms of Use and Notice of Rights

This book is protected by copyright law. Unauthorized copying, sharing, or selling of this book, in whole or in part, is prohibited.

You may not, without the prior written permission of the author:

- Reproduce, copy, or scan this book, in print or digital form.
- Share, upload, or distribute it, including through file-sharing sites, messaging groups, cloud drives, and social media.
- Sell, resell, rent, lend for profit, or otherwise offer it for payment, alone or bundled with other products.
- Translate, adapt, or republish it, or present its text, structure, or exercises as your own work.
- Use it to train, fine-tune, or build datasets for machine-learning or AI systems.

Brief quotations in reviews and educational discussions are allowed, as long as the book and its author are credited.

If you bought or downloaded this book from anyone other than the author or an authorized seller, your copy is unauthorized. Please buy a legitimate copy. Purchases support the work that goes into the next edition.

Code Examples

You may use the code examples in your own programs and learning projects. You may not republish them in bulk, as part of a competing book, course, or tutorial.

Reporting Piracy

To report an unauthorized copy, or to request permission to use material from this book, contact the author at crafty_shadow@proton.me.

Disclaimer

The author has worked to make the information in this book accurate and complete. The book and its code are provided "as is," without warranty of any kind, express or implied. The author is not liable for any direct, indirect, incidental, or consequential damages arising from the use of the code, software, or ideas in this book.

Trademarks

Minecraft is a trademark of Mojang Synergies AB and Microsoft. Python is a trademark of the Python Software Foundation. All other product and company names are the property of their respective owners. This book is an independent work. It is not affiliated with, endorsed by, or sponsored by any of them.

About the Author

About the Author

Md Nasrullah

Developer and Cybersecurity Researcher
`Python • Offensive Security • OS Development`

Md Nasrullah is a developer and cybersecurity researcher. He works mainly in Python, offensive security, and low-level systems, and he wrote ***The Curious Coder — Part II: Beyond the Basics*** for readers who have completed **The Curious Coder — Part I: Python Foundations** or already know its core material. This book builds on that foundation rather than starting from zero.

He believes you learn programming by building things and breaking them. That belief shapes this book: every chapter asks you to run code, change it, and see what happens.

His own work covers:

- **Low-level systems:** system utilities and operating system development.
- **Cybersecurity:** network security testing, penetration testing, and vulnerability assessment.
- **Application engineering:** practical Python tools, scripts, and modular design.
- **Learning by teaching:** turning hard ideas into small examples a beginner can run.

What connects these areas is one question: how does this actually work? Nasrullah asks it all the way down, from the application to the hardware beneath it. This book is built to help you start asking it too.

The Curious Coder Philosophy

1. **Learn by building.** Understand an idea by using it in working software.
2. **Look underneath.** Syntax is only the surface. Ask what the machine is really doing.
3. **Experiment without fear.** A broken program is information, not failure.
4. **Stay curious.** Keep asking deeper questions, and keep building.

Find the Author

Questions, ideas, and updates on new editions are welcome.

- **YouTube (programming, security, and tech):** crafty_security
- **YouTube (Minecraft):** crafter_2505
- **Instagram:** crafter_2505
- **TikTok:** crafter_2505
- **Discord:** crafter_2505yt

Table of contents

11. Loops and Repetition 7

12. Controlling Loops and Solving Repetitive Problems 19

13. Lists and Working with Sequences 33

14. Processing Collections of Data 59

15. Tuples and Sets 76

16. Dictionaries and Key-Value Data 93

17. Functions and Reusable Logic 115

18. Function Design and Variable Scope 132

19. Exceptions, Debugging, and Input Validation 153

20. Files and Persistent Data 176

21. Modules, Imports, and Organizing Projects 201

22. Comprehensions and Concise Data Processing 222

23. Capstone, Build a Task Management Application 241

Beyond the Basics

You know the basics of Python. You can create variables, work with different data types, receive input, use operators, and make decisions with conditions.

But writing a program that performs a few operations is different from building one that can handle an entire task.

Consider a task management application. It needs to store multiple tasks, let users add new ones, find specific tasks, update their status, and save everything so the information is still available the next time the program runs.

You already have some of the knowledge needed to build it. Now we will learn the tools that bring those pieces together.

The Curious Coder — Part II: Beyond the Basics continues where Python Foundations ended. We will move beyond individual statements and start building programs with more structure, more data, and more responsibility.

Part I introduced you to Python and its basic building blocks. Part II focuses on using those building blocks together.

You will learn loops to repeat instructions, collections to work with groups of values, and functions to organize reusable pieces of code. Then we will work with dictionaries, exceptions, files, modules, and comprehensions.

Each topic adds something new to the way you write programs. You will begin with small examples, then combine concepts to solve larger problems.

The examples continue to use familiar situations, including Minecraft PvP players, ranks, tournaments, and game-related data. As before, the goal is to understand the Python behind each example, not to memorize a collection of programs.

By the end, you will build a Task Management Application. It will bring together the concepts from the book, including functions, collections, file storage, and input validation. You will be able to add, view, edit, complete, delete, search, and filter tasks.

How to Use This Book

You learn programming by writing code. Reading helps you understand an idea, but testing it yourself is how you learn to use it.

Work through each chapter in five steps:

1. **Read.** Understand the new idea and why it exists. Pay attention to the problem it solves.
2. **Run.** Type the examples yourself and observe what happens. Follow the code instead of skipping straight to the output.
3. **Experiment.** Change values, modify instructions, and predict the result before running the program again.

4. **Solve.** Complete the exercises without looking at the answers first. Try to work through the problem yourself.
5. **Reflect.** Explain the concept in your own words. Think about how you could use it in another program.

Some topics will take more time than others. When something feels difficult, return to the smaller examples and build your understanding one step at a time.

A Note About Mistakes

Errors are part of programming. They are not a sign that you are bad at it.

As programs become larger, there are more places where something can go wrong. A loop might repeat too many times. A function might receive an unexpected value. A file might not contain the information your program expects.

When an error appears, read the message and inspect the relevant code. Change one thing at a time and run the program again.

You will also learn how to handle certain errors inside your programs. This helps your applications respond to unexpected situations instead of stopping whenever something goes wrong.

The goal is to understand your mistakes and learn how to fix them.

Before You Begin

This book assumes you have completed **The Curious Coder — Part I: Python Foundations**. You should already have Python and a code editor installed.

Keep Part I nearby if you need to revisit an earlier topic. Some chapters will use concepts such as conditions, strings, operators, and input without explaining them from the beginning again.

You do not need to rush through the chapters. Take time to type the examples, experiment with the code, and complete the exercises.

And when you reach the final project, try to build each part yourself before looking at the solution.

Let's go beyond the basics.

11. Loops and Repetition

Imagine a Minecraft tournament with five rounds. At the beginning of each round, the program needs to display the round number.

One way to do that is to write five separate instructions:

```
print("Round 1")
print("Round 2")
print("Round 3")
print("Round 4")
print("Round 5")
```

It works, but the code gets longer every time you add another round. A tournament with 100 rounds would need 100 print statements.

There is a better way. Python can repeat instructions for you.

Your First Loop

A loop is a control structure that runs a block of code more than once.

```
for attempt in range(3):
    print("Checking tournament entry...")
```

Output:

```
Checking tournament entry...
Checking tournament entry...
Checking tournament entry...
```

The `print()` instruction appears once but runs three times.

Python has two main types of loops:

- `while` loops repeat as long as a condition is true.
- `for` loops repeat for each value in a sequence.

The difference becomes clearer when you use them.

The While Loop

A `while` loop repeats its code while a condition remains `True`.

```
while condition:
    # Code to repeat
```

Python checks the condition before entering the loop. If it is true, the indented code runs. After that, Python checks the condition again.

Here's a program that displays three tournament rounds:

```
round_number = 1

while round_number <= 3:
    print(f"Round {round_number}")
    round_number += 1
```

Output:

```
Round 1
Round 2
Round 3
```

The variable `round_number` starts at `1`. Python checks whether it is less than or equal to `3`. Since that condition is true, the loop runs.

After displaying the round, this instruction increases the variable:

```
round_number += 1
```

The `+=` operator adds a value to the variable's existing value. Here, it increases `round_number` by one.

The condition is checked again with the new value. This happens until `round_number` becomes `4`. At that point, `round_number <= 3` is false, so Python leaves the loop.

Updating a Variable Inside a Loop

The changing value is what allows this loop to finish.

Compare the next program with the earlier loop. Notice anything missing?

```
round_number = 1

while round_number <= 3:
    print(f"Round {round_number}")
```

`round_number` never changes. It stays at `1`, so the condition remains true and Python keeps printing the same round.

This is an **infinite loop**.

I once spent a long time checking a loop that wouldn't stop, only to find that I had forgotten to update its counter. It's an easy mistake to make.

If your program appears frozen, check whether the loop can actually reach its stopping condition. In the examples in this chapter, that happens through an assignment inside the loop.

When Should You Use a While Loop?

Use `while` when you want repetition to depend on a condition rather than a fixed number of repetitions.

For example, a tournament countdown can continue until it reaches zero.

Before running this program, predict how many numbers it will display.

```
countdown = 5

while countdown > 0:
    print(countdown)
    countdown -= 1

print("Tournament starting!")
```

Output:

```
5
4
3
2
1
Tournament starting!
```

The loop runs while `countdown` is greater than zero. The `-=` operator subtracts one from its current value after every repetition.

When the countdown reaches zero, the loop ends and the final message is printed.

The For Loop

A `for` loop repeats its code for each value in a sequence.

```
for variable in sequence:
    # Code to repeat
```

Python takes the next value from the sequence, assigns it to the loop variable, and runs the indented code. It continues until there are no more values.

We can use `range()` to generate the sequence.

```
for number in range(5):
    print(number)
```

Output:

```
0
1
2
3
4
```

The loop runs five times. The variable `number` receives `0`, then `1`, then `2`, and so on until it receives `4`.

Why does it begin at zero? And why does it stop at four?

To understand that, we need to look at `range()`.

Understanding range()

The `range()` function generates a sequence of integers. It is commonly used with `for` loops when you know how many times something needs to repeat.

It accepts one, two, or three arguments.

One Argument

```
range(stop)
```

With one argument, the sequence begins at `0` and stops before the specified value.

```
for round_number in range(5):
    print(f"Round {round_number + 1}")
```

Output:

```
Round 1
Round 2
Round 3
Round 4
Round 5
```

The generated values are `0` through `4`. We add one when displaying them because tournament rounds begin at `1`.

The two-argument form, coming next, lets you start at `1` directly.

Two Arguments

```
range(start, stop)
```

The first argument sets the starting value. The second sets where the sequence stops.

For example, a tournament might display rounds five through eight:

```
for round_number in range(5, 9):
    print(f"Round {round_number}")
```

Output:

```
Round 5
Round 6
Round 7
Round 8
```

The starting value is included, but the stopping value is not. That is why `9` produces a final round numbered `8`.

Three Arguments

```
range(start, stop, step)
```

The third argument controls how much the value changes each time.

Suppose a game displays a milestone every five levels:

```
for level in range(5, 26, 5):
    print(f"Level milestone: {level}")
```

Output:

```
Level milestone: 5
Level milestone: 10
Level milestone: 15
Level milestone: 20
Level milestone: 25
```

The sequence begins at `5` and increases by `5` each time. It stops before reaching `26`.

A negative step lets you count backwards:

```
for number in range(5, 0, -1):
    print(number)
```

Output:

```
5
4
3
2
1
```

Here, the value decreases by one on each repetition.

When range() Produces Nothing

A `range()` sequence can be empty. For example:

```
for number in range(0):
    print(number)

print("Program finished.")
```

Output:

```
Program finished.
```

The loop body never runs because `range(0)` contains no values. Python does not raise an error. The same thing happens with `range(5, 5)`, because the starting and stopping values are equal.

If a loop unexpectedly does nothing, check the values passed to `range()`.

Using a For Loop with Calculations

Loops can do more than display numbers. They can update variables and calculate results.

Suppose a player earns a different number of wins in five tournament rounds:

Round	Wins
1	1
2	2
3	3
4	4
5	5

We want to calculate the total.

You could add the values manually, but a loop can do the calculation:

```
total_wins = 0

for wins in range(1, 6):
    total_wins += wins

print(f"Total wins: {total_wins}")
```

Output:

```
Total wins: 15
```

The variable `total_wins` starts at zero. Each time the loop runs, the current value of `wins` is added to it.

Before looking at the table, try tracing the loop by hand. What will `total_wins` contain after each repetition?

Repetition	wins	total_wins
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

The variable `total_wins` is an **accumulator**. An accumulator stores a result that changes as a program processes values.

When a calculation seems confusing, write down the variable's value after each repetition. Tracing the values on paper can make it much easier to find where a result comes from.

While vs. For

Both loops repeat instructions, but they control repetition differently.

Here are two programs that display the numbers from `1` through `5`.

Using while:

```
number = 1

while number <= 5:
    print(number)
    number += 1
```

Using for:

```
for number in range(1, 6):
    print(number)
```

Both produce:

```
1
2
3
4
5
```

The `while` version needs a variable and a condition that you update. The `for` version gets its values from `range()`.

Use `while` when the program needs to keep checking a condition. Use `for` when you have a sequence of values to process or a known number of repetitions.

Common Mistakes

The following examples show loop mistakes that are easy to make. Read each code block and see if you can spot the problem before reading the explanation.

Forgetting to Update the Variable

```
number = 1

while number <= 5:
    print(number)
```

The condition checks `number`, but nothing changes its value. It stays at `1`, so the loop never finishes.

The loop needs an update such as:

```
number += 1
```

Using the Wrong range() Stop Value

```
for number in range(1, 5):
    print(number)
```

Output:

```
1
2
3
4
```

If you expected the loop to display `5`, the stopping value is the problem. `range()` excludes its stop argument.

Use `range(1, 6)` to include `5`.

Updating the Wrong Variable

A program can have more than one variable, and it is easy to update the wrong one.

The extra `and checks < 3` condition below limits the demonstration to three repetitions. The `and` operator requires both conditions to be true. You'll study this operator in a later chapter.

```
round_number = 1
next_round = 2
checks = 0

while round_number <= 3 and checks < 3:
    print(f"Round {round_number}")
    next_round += 1  # BUG: this should update round_number
    checks += 1
```

Output:

```
Round 1
Round 1
Round 1
```

The problem is that the program changes `next_round`, but the loop condition checks `round_number`. The value of `round_number` stays at `1`.

The `checks` variable only limits this demonstration. Without that limit, the loop would keep running because `round_number` never changes.

Incorrect Indentation

```
for number in range(3):
    print(number)

print("Finished")
```

Output:

```
0
1
2
Finished
```

The final `print()` statement is not indented, so it is outside the loop. It runs once after the loop finishes.

If it were indented, it would run during every repetition instead.

Practice Exercises

Write each program in your editor. Pay attention to the loop condition, the values produced by `range()`, and the variables that change.

Exercise 1: Number Display

Write a `for` loop that displays the numbers from `1` through `5`.

Expected output:

```
1
2
3
4
5
```

Exercise 2: Countdown

Write a `while` loop that counts backwards from `5` to `1`.

Expected output:

```
5
4
3
2
1
```

Exercise 3: Multiplication Table

Write a program that displays the multiplication table for `7`, from `7 * 1` through `7 * 10`.

Expected output:

```
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
7 * 10 = 70
```

Exercise 4: Calculate a Total

Write a program that uses a loop to calculate the sum of all integers from `1` through `10`.

Expected output:

```
Sum: 55
```

Answers

Exercise 1: Number Display

```
for number in range(1, 6):
    print(number)
```

Output:

```
1
2
3
4
5
```

Exercise 2: Countdown

```
number = 5

while number >= 1:
    print(number)
    number -= 1
```

Output:

```
5
4
3
2
1
```

Exercise 3: Multiplication Table

```
for multiplier in range(1, 11):
    print(f"7 * {multiplier} = {7 * multiplier}")
```

Output:

```
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
7 * 9 = 63
7 * 10 = 70
```

Exercise 4: Calculate a Total

```
total = 0

for number in range(1, 11):
    total += number

print(f"Sum: {total}")
```

Output:

```
Sum: 55
```

12. Controlling Loops and Solving Repetitive Problems

In the previous chapter, we learned how loops repeat instructions. A `for` loop can process a sequence of values, while a `while` loop can keep running as long as a condition remains true.

But sometimes, repeating instructions isn't enough. We also need to control *when* a loop stops and which parts of the loop should run.

Think about a Minecraft tournament. A player might leave the match early, a round might skip a player who is unavailable, or a search might stop as soon as the correct username is found.

Python gives us three tools for controlling loops:

- `break` stops a loop immediately.
- `continue` skips the rest of the current iteration.
- `else` can run after a loop finishes without being stopped by `break`.

Let's see how each one works.

The break Statement

The `break` statement immediately ends the loop it belongs to. Python then continues with the first statement after that loop.

Suppose we're checking tournament rounds. The tournament normally has five rounds, but a player has decided to leave after the third round.

```
for round_number in range(1, 6):
    print(f"Round {round_number}")

    if round_number == 3:
        print("The player has left the tournament.")
        break

print("Tournament check finished.")
```

Output:

```
Round 1
Round 2
Round 3
The player has left the tournament.
Tournament check finished.
```

The loop was supposed to run five times, but `break` stopped it when `round_number` reached `3`.

Notice that the final `print()` still ran. It is outside the loop, so ending the loop does not skip it.

Using break to Find Something

Another common use of `break` is searching through a collection.

Suppose we have a list of player usernames and want to find a particular player.

```
players = ["crafter_2505", "block_builder", "redstone_dev"]
target = "block_builder"

for player in players:
    print(f"Checking {player}")

    if player == target:
        print("Player found!")
        break
```

Output:

```
Checking crafter_2505
Checking block_builder
Player found!
```

Once Python finds the target, there is no reason to keep checking the remaining players. The `break` statement stops the search.

Without it, Python would continue through the rest of the list even though the result was already known.

The continue Statement

While `break` ends the entire loop, `continue` only skips the current iteration.

The loop itself keeps running. Python moves to the next iteration instead of executing the remaining statements in the current one.

Suppose we're processing player scores, but we only want to display positive scores. Zero and negative values should be skipped.

```
scores = [12, 0, 8, -2, 15]

for score in scores:
    if score <= 0:
        continue

    print(f"Score: {score}")
```

Output:

```
Score: 12
Score: 8
Score: 15
```

When Python reaches `0`, the condition is true, so `continue` skips the `print()` statement. The same thing happens with `-2`.

But the loop does not stop. It moves on and processes `15`.

Here's the difference:

- `break` ends the loop completely.
- `continue` skips only the current iteration.

That distinction matters when you want to ignore certain values without stopping the entire process.

Using else with Loops

You've already used `else` with `if` statements. But Python also allows an `else` block after a `for` or `while` loop.

A loop's `else` block runs when the loop finishes normally. If the loop is stopped by `break`, the `else` block is skipped.

Unlike an `if` statement's `else`, which handles the condition being false, a loop's `else` depends on whether the loop ended without executing `break`.

Searches are a natural fit because a search either finds something or runs out of items to check.

Suppose we want to check whether a username exists in a player list.

```
players = ["crafter_2505", "block_builder", "redstone_dev"]
target = "pixel_runner"

for player in players:
    if player == target:
        print("Player found!")
        break
else:
    print("Player not found.")
```

Output:

```
Player not found.
```

None matches `pixel_runner`, so the loop finishes without executing `break`. The `else` block then runs.

Now change the target:

```
target = "block_builder"
```

The loop finds the player and executes `break`. That means the loop's `else` block is skipped.

How Is This Different from a Boolean Flag?

You can solve the same problem using a Boolean variable. We used Boolean values in Part I to represent `True` and `False`.

```
players = ["crafter_2505", "block_builder", "redstone_dev"]
target = "pixel_runner"

found = False

for player in players:
    if player == target:
        found = True
        break

if found:
    print("Player found!")
else:
    print("Player not found.")
```

Output:

```
Player not found.
```

Both approaches work.

The Boolean flag makes the result explicit, which can be useful when you need to use it later in the program. The loop's `else` can make a simple search shorter.

You don't have to use loop `else` every time. Choose the approach that makes your code easier to understand.

Building a Number Guessing Game

The next program uses loop control in a small guessing game.

The program will choose a secret number. The player gets three attempts to guess it. After each incorrect guess, the program gives a hint.

We'll use `break` to stop the game when the player guesses correctly. The loop's `else` will handle the situation where all attempts are used without a correct guess.

First, here's the basic structure. The variables will be defined before the loop in the complete program below.

```
while attempts < max_attempts:
    # Ask for a guess and check it
    ...
else:
    # This runs if the loop finishes without break
    ...
```

The `else` belongs to the `while` loop, not to an `if` statement inside it. It runs only when the loop condition becomes false without the loop being stopped by `break`.

Here's the complete program:

```
import random

secret_number = random.randint(1, 10)
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    guess_text = input("Guess the number (1-10): ")
    guess = int(guess_text)
    attempts += 1

    if guess == secret_number:
        print("Correct! You guessed the number.")
        break
    elif guess < secret_number:
        print("Too low.")
    else:
        print("Too high.")
else:  # Runs only if the while loop finishes without break
    print("No attempts remaining.")

print(f"The secret number was {secret_number}.")
```

The `random.randint(1, 10)` function from Python's `random` module chooses a random integer from `1` to `10`, including both endpoints.

A sample interaction might look like this:

```
Guess the number (1-10): 4
Too low.
Guess the number (1-10): 9
Too high.
Guess the number (1-10): 7
Correct! You guessed the number.
The secret number was 7.
```

The secret number is random, so your interaction will probably be different.

There are two steps in this line:

```
guess = int(input("Guess the number (1-10): "))
```

First, `input()` reads what the player types as text. Then, `int()` converts that text into an integer.

We can write those steps separately if the nested version looks unfamiliar:

```
guess_text = input("Guess the number (1-10): ")
guess = int(guess_text)
```

The `attempts` variable increases after each guess. Once it reaches `max_attempts`, the loop condition becomes false.

If the player guesses correctly, `break` stops the loop immediately, so the loop's `else` does not run. If all three guesses are incorrect, the loop finishes normally and the `else` block runs.

Building a Menu-Driven Calculator

Many programs need to keep running until the user chooses to exit. A calculator is a good example.

A user might perform several calculations before closing the program. We don't want to restart the program after every operation.

One way to create this kind of loop is with `while True`.

```
while True:
    # Instructions that repeat
```

The condition is always `True`, so this loop does not stop on its own. We need a `break` statement to end it when the user chooses to exit.

Here's a small calculator with addition, subtraction, and an exit option:

```
while True:
    print("\n--- Calculator ---")
    print("1. Addition")
    print("2. Subtraction")
    print("3. Exit")

    choice = input("Choose an option: ")

    if choice == "1":
        first = float(input("Enter the first number: "))
        second = float(input("Enter the second number: "))
        print(f"Result: {first + second}")

    elif choice == "2":
        first = float(input("Enter the first number: "))
        second = float(input("Enter the second number: "))
        print(f"Result: {first - second}")
```

```
    elif choice == "3":
        print("Goodbye!")
        break

    else:
        print("Invalid option. Try again.")

print("Calculator closed.")
```

A sample interaction:

```
--- Calculator ---
1. Addition
2. Subtraction
3. Exit
Choose an option: 1
Enter the first number: 4
Enter the second number: 5
Result: 9.0

--- Calculator ---
1. Addition
2. Subtraction
3. Exit
Choose an option: 7
Invalid option. Try again.

--- Calculator ---
1. Addition
2. Subtraction
3. Exit
Choose an option: 3
Goodbye!
Calculator closed.
```

The menu keeps appearing because it is inside the loop. An invalid option displays a message, but it doesn't stop the program. Choosing option `3` executes `break`, ending the loop.

You can use this same pattern in many programs that need to keep accepting commands until the user decides to stop.

Common Loop-Control Mistakes (and How to Fix Them)

One way to understand loop control is to look at what happens when the wrong statement is used. These mistakes can be difficult to spot if you only read the code, so pay attention to which statements Python skips.

1. Using break Instead of continue

Suppose we want to skip the score `0` but keep processing the remaining scores.

```
scores = [12, 0, 8, 15]

for score in scores:
    if score == 0:
        break  # BUG: this stops the entire loop

    print(score)
```

Output:

```
12
```

The loop stops as soon as it reaches `0`. The values `8` and `15` are never processed.

If we want to skip only the current value, we should use `continue`:

```
scores = [12, 0, 8, 15]

for score in scores:
    if score == 0:
        continue

    print(score)
```

Output:

```
12
8
15
```

2. Expecting a Loop's else to Run After break

Remember, a loop's `else` block runs only when the loop finishes without `break`.

```
for number in range(1, 6):
    print("Checking", number)

    if number == 3:
        break
else:
    print("Loop finished.")
```

Output:

```
Checking 1
Checking 2
Checking 3
```

The loop ran, but the `else` block did not print anything. That's because `break` stopped the loop when `number` reached `3`.

If we remove the `break`, the loop finishes normally and prints `Loop finished.`

3. Using continue Before Updating a while Loop

This mistake is especially easy to make with `while` loops.

Look at this example:

```
count = 0

while count < 5:
    if count == 2:
        continue  # BUG: skips the count += 1 below, so count stays at 2

    print(count)
    count += 1
```

Do not run this example. It contains an infinite-loop bug.

When `count` reaches `2`, Python executes `continue`. That skips the remaining statements in the loop, including `count += 1`.

So `count` stays at `2`. The condition `count < 5` remains true, and the loop keeps repeating.

We can fix the problem by saving the current value and updating `count` before reaching `continue`:

```
count = 0

while count < 5:
    current_count = count  # save the value for the check below
    count += 1             # increment before continue can skip it

    if current_count == 2:
        continue

    print(current_count)
```

Output:

```
0
1
3
4
```

We use `current_count` because `count` must be increased before `continue` can run. Saving its old value lets us still check and print the number that belonged to the current iteration.

Practice Exercises

Use the loop-control statements from this chapter to solve the following problems. You can reuse the examples above as a starting point.

Exercise 1: Skip a Value

You have a sequence of numbers from `1` to `7`.

Write a `for` loop that prints every number except `3`. Use `continue` to skip that value.

Expected output:

```
1
2
4
5
6
7
```

Exercise 2: Search for a Player

Create a list containing these usernames:

```
players = ["crafter_2505", "block_builder", "redstone_dev"]
```

Ask the user to enter a username. Search the list and use `break` when you find a match.

If the username is found, print:

```
Player found!
```

Otherwise, print:

```
Player not found.
```

Use a loop's `else` block to handle the case where the username does not exist.

Remember that `input()` returns the text entered by the user. You can compare that text directly with the usernames in the list.

Exercise 3: Stop a Menu

Create a menu that keeps running until the user enters `x`.

The menu should have these options:

- `a` displays `You selected option A.`
- `b` displays `You selected option B.`
- `x` displays `Menu closed.` and stops the loop.
- Any other input displays `Invalid option.`

Use `while True` and `break`.

Exercise 4: Improve the Guessing Game

Write a number guessing game that:

1. Stores a secret number in a variable.
2. Gives the player three attempts.
3. Displays `Too low.` or `Too high.` after an incorrect guess.
4. Stops immediately when the player guesses correctly.
5. Displays `No attempts remaining.` if all attempts are used.

Use `break` and a loop's `else` block.

Hint: The loop needs to stop in two different ways: a correct guess, or running out of attempts. How can you distinguish those?

Answers

Exercise 1: Skip a Value

```
for number in range(1, 8):
    if number == 3:
        continue

    print(number)
```

Output:

```
1
2
4
5
6
7
```

When `number` equals `3`, `continue` skips the `print()` statement. The loop then moves to `4`.

Exercise 2: Search for a Player

```
players = ["crafter_2505", "block_builder", "redstone_dev"]
target = input("Enter a username: ")

for player in players:
    if player == target:
        print("Player found!")
        break
else:
    print("Player not found.")
```

Example interaction when the player exists:

```
Enter a username: redstone_dev
Player found!
```

Example interaction when the player does not exist:

```
Enter a username: pixel_runner
Player not found.
```

A close but non-matching username also produces `Player not found.`:

```
Enter a username: redstone
Player not found.
```

The comparison checks for an exact match. The `break` statement stops the search when a match is found. If the loop checks every username without finding one, its `else` block runs.

Exercise 3: Stop a Menu

```
while True:
    print("\n--- Menu ---")
    print("a. Option A")
    print("b. Option B")
    print("x. Exit")

    choice = input("Choose an option: ").lower()

    if choice == "a":
        print("You selected option A.")
    elif choice == "b":
        print("You selected option B.")
    elif choice == "x":
        print("Menu closed.")
        break
    else:
        print("Invalid option.")
```

Example interaction:

```
--- Menu ---
a. Option A
b. Option B
x. Exit
Choose an option: a
You selected option A.

--- Menu ---
```

```
a. Option A
b. Option B
x. Exit
Choose an option: z
Invalid option.

--- Menu ---
a. Option A
b. Option B
x. Exit
Choose an option: x
Menu closed.
```

The loop repeats after options `a`, `b`, or an invalid input. It ends only when the user enters `x`.

Exercise 4: Improve the Guessing Game

The exercise uses a fixed secret number (7) so the sample interactions are predictable. The guessing game earlier in the chapter uses `random.randint()` to choose a different number each time.

```
secret_number = 7
attempts = 0
max_attempts = 3

while attempts < max_attempts:
    guess_text = input("Guess the number (1-10): ")
    guess = int(guess_text)
    attempts += 1

    if guess == secret_number:
        print("Correct! You guessed the number.")
        break
    elif guess < secret_number:
        print("Too low.")
    else:
        print("Too high.")
else:
    print("No attempts remaining.")
```

Example interaction when the player guesses correctly:

```
Guess the number (1-10): 4
Too low.
Guess the number (1-10): 9
Too high.
Guess the number (1-10): 7
Correct! You guessed the number.
```

Example interaction when the player uses all attempts:

```
Guess the number (1-10): 2
Too low.
Guess the number (1-10): 9
Too high.
Guess the number (1-10): 5
Too low.
No attempts remaining.
```

The first interaction ends through `break`, so the loop's `else` is skipped. In the second interaction, the loop reaches its attempt limit without `break`, so the `else` block runs.

END OF SAMPLE

Enjoyed this? There is much more ahead.

The full book is 275 pages and continues with:

13 Lists and Working with Sequences
14 Processing Collections of Data
15 Tuples and Sets
16 Dictionaries and Key-Value Data
17 Functions and Reusable Logic
18 Function Design and Variable Scope
19 Exceptions, Debugging, and Input Validation
20 Files and Persistent Data
21 Modules, Imports, and Organizing Projects
22 Comprehensions and Concise Data Processing
23 Capstone: Build a Task Management Application

GET IN TOUCH

Questions, ideas, and updates on new editions are welcome.

Email: crafty_shadow@proton.me
YouTube (programming, security, and tech): crafty_security